UNIVERSIDAD DE LA SALLE



Unidad 3

Análisis de resultados y mejora continua

Autor: Alvaro Javier Hernández Rueda

Contenido

Introducción 6

Objetivo de la lección 7

1. Métricas y cuadros de mando 8

a. KPIs de QA (defect density, escape rate, MTTR) 8

b. Métricas DORA y fiabilidad operativa 9

c. Construcción de dashboards ejecutivos (Grafana / Power BI) 10

2. Ciclo de vida del defecto y triage 12

a. Flujo New → Closed (severity/priority) 12

I. Estados mínimos y transiciones recomendadas 13

II. Severity vs Priority (definiciones operativas) 13

III. Plantilla de reporte y criterios de calidad 14

IV. Políticas de triage y SLA de atención (ejemplo formativo) 14

V. Criterios de entrada y cierre (DoR/DoD) para defectos 14

VI. Mapeo a herramientas (buenas prácticas) 14

VII. Errores frecuentes (y cómo evitarlos) 15

b. Roles y dinámica de la reunión de *triage* 15

I. Quién participa y con qué rol (RACI del *triage*) 15

II. Dinámica sugerida (agenda de 30–45 min) 16

III. Métricas operativas del *triage* (para tu tablero) 17

IV. Buenas prácticas (checklist rápido) 17

V. Antipatrones frecuentes 18

c. Gestión de regresiones y hotfixes 18

I. Política de regresiones (mínimo viable y repetible) 18

II. Hotfix: flujo operativo recomendado (con variantes de *branching*) 19

III. *Checklists* prácticos 20

IV. Buenas prácticas 20

V. Antipatrones (y cómo evitarlos) 21

3. Análisis causa-raíz 21

a. 5 Whys, Diagrama Ishikawa, Pareto 21

I. 5 Whys (cinco porqués) 21

II. Diagrama Ishikawa (espina de pescado / causa-efecto) 22

III. Pareto (80/20) y gráfico de Pareto 23

b. Técnicas de blameless *post-mortem* 24

I. Principios de un *post-mortem* sin culpa 25

II. Estructura mínima del documento (plantilla sugerida) 25

III. Dinámica de la reunión (facilitación práctica, 45–60 min) 26

IV. Ejemplos de acciones eficaces (evita los *antipatrones*) 26

V. Métricas de efectividad del *post-mortem* 26

VI. Integración con cultura y procesos 27

4. Integración QA en CI/CD 27

a. Quality Gates en Jenkins/GitLab 27

I. Jenkins: quality gates en la pipeline 27

II. GitLab: quality gates en merge requests 30

III. Diseñar tu *quality gate* (*checklist* recomendado) 32

IV. Antipatrones (y cómo evitarlos) 32

b. Política de fail-fast y *feature toggles* 32

I. ¿Qué entendemos por fail-fast? 33

II. ¿Qué son los feature toggles y por qué importan? 33

III. Implementación práctica 33

IV. Política integrada 35

V. Antipatrones frecuentes (y su corrección) 36

VI. Guía de despliegue seguro con flags (paso a paso) 36

5. Mejora continua y reporting 37

a. Kaizen & retros del equipo QA 37

I. Fundamentos de Kaizen aplicados a QA 37

II. Retrospectivas (estructura y dinámica) 37

III. ¿Cuándo usar qué ?(Kaizen, Retro, A3, Kata) 38

IV. Tablero Kaizen (operativizar la mejora) 38

V. Métricas de efectividad (mejora y retro) 38

VI. Buenas prácticas (*checklist*) 38

VII. Antipatrones (y cómo evitarlos) 39

b. Comunicación de hallazgos a stakeholders 39

I. ¿Qué debe entregarse y con qué formato? 39

II. Mapear *stakeholders* y mensajes (quién necesita qué) 40

III. Estructura de un mensaje efectivo (guion de 6 pasos) 40

IV. Visualización y dashboards para *stakeholders* 41

V. Cadencia y canales (sincrónico + asíncrono) 41

VI. Ejemplos 41

VII. Antipatrones (y cómo evitarlos) 42

c. Roadmap de optimización incremental 42

I. Poner el piso de medición (línea base y *north star*) 42

II. Mapear el flujo y hallar el cuello de botella 43

III. Priorizar con economía de decisiones 43

IV. Diseñar mejoras como experimentos PDSA 43

V. Ejecutar con despliegues seguros (canary/flags) y *gates* 43

VI. Medir el efecto y hacer visible el progreso 44

VII. Cadencia de gobierno (ligera y efectiva) 44

VIII. Ejemplo de roadmap (90 días) 44

IX. Antipatrones (y cómo evitarlos) 45

Revisa tus aprendizajes 45

Lecciones aprendidas 48

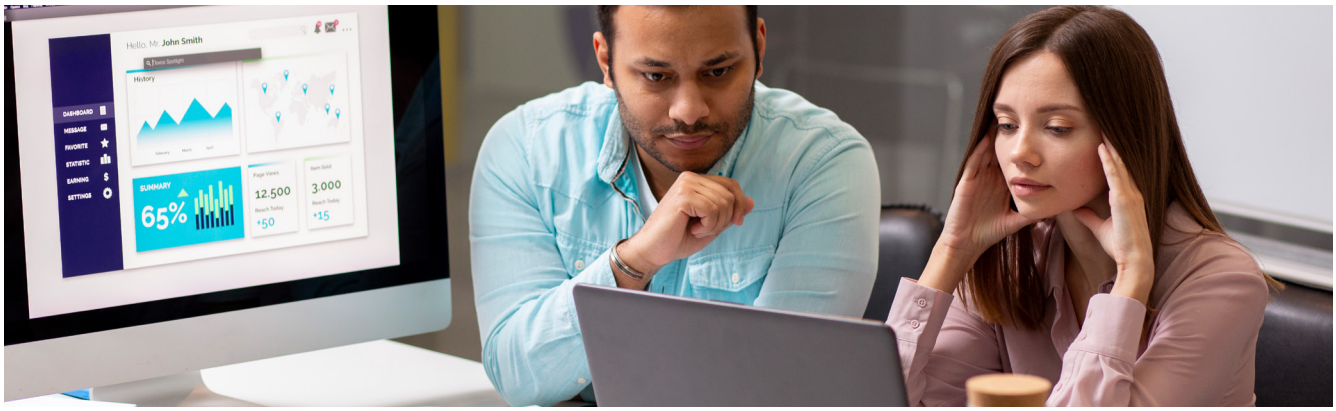
Referencias bibliográficas 49

Introducción



En esta unidad, analizamos resultados y promovemos la mejora continua a partir de datos confiables y decisiones bien informadas. Comenzarás comprendiendo qué medir y cómo: *KPIs de QA* (como *defect density, escape rate* y *MTTR*), las métricas DORA y su relación con la fiabilidad operativa. Luego, construirás cuadros de mando ejecutivos (en Grafana o Power BI) que muestren percentiles, tendencias y cumplimiento de SLO/SLA de un vistazo. Pasarás al terreno operativo con el ciclo de vida del defecto (New→Closed), la reunión de *triage* y la gestión de regresiones y *hotfixes*. Para explicar por qué ocurren los problemas y cómo evitarlos, aplicarás 5 Whys, Ishikawa y Pareto, además de practicar *post-mortems* sin culpa. Finalmente, integrarás la calidad en el flujo de entrega con quality gates en Jenkins/GitLab, políticas fail-fast y feature toggles, cerrando con prácticas de Kaizen, retrospectivas, comunicación de hallazgos y un *roadmap* de optimización incremental. Al finalizar, podrás responder: ¿cómo diseño un tablero y un proceso de decisión que reduzcan el escape *rate* y el MTTR sin sacrificar la frecuencia de despliegue?

Objetivo de la lección



Al finalizar esta lección, el estudiante será capaz de:

- **Definir y calcular** KPI de QA (*defect density, escape rate*, MTTR) y **explicar su uso** para evaluar calidad del producto y del proceso.
- **Interpretar y usar** las métricas **DORA** (DF, LT, CFR, MTTR) para equilibrar **velocidad y estabilidad** en la entrega de software.
- **Diseñar y construir *dashboards* ejecutivos** (Grafana / Power BI) que muestren percentiles, tendencias y **cumplimiento de SLO/SLA**, con definiciones operativas visibles.
- **Estandarizar** el **ciclo de vida del defecto (New → Closed)**, distinguir **severity vs. priority**, y **facilitar** reuniones de ***triage*** efectivas con roles y acuerdos claros.
- **Aplicar** técnicas de **análisis causa-raíz** (5 Whys, Ishikawa, Pareto) y **conducir** *post-mortems* **blameless** con acciones SMART y seguimiento.
- **Integrar QA en CI/CD** configurando **quality gates** (cobertura, estática, seguridad), políticas **fail-fast** y **feature toggles** para *rollouts* seguros.
- **Implementar** prácticas de **mejora continua** (Kaizen, retros) y **comunicar** hallazgos a *stakeholders* usando **BLUF** y tableros de una página.
- **Elaborar** un **roadmap de optimización incremental** (90 días) basado en métricas, SLO/*error-budget* y foco en cuellos de botella.

1. Métricas y cuadros de mando



a. KPIs de QA (defect density, escape rate, MTTR)

Estos indicadores miden la **calidad del producto** y la **efectividad del proceso**. Para que sean comparables, defínelos con precisión (universo, ventana, fórmula) y valida que las fuentes de datos sean consistentes (estándares ISO/IEC 25023; metodología IEEE-1061; guías SRE sobre incidentes).

Defect density (densidad de defectos). Número de **defectos válidos** por unidad de tamaño del artefacto/versionado (p. ej., **defectos/KLOC o defectos/FP**).

$$Defect\ density = \frac{\#\ defectos\ válidos}{tamaño\ (KLOC,\ FP,\ \ldots)}$$

Recomendaciones: usa una sola unidad para comparar módulos; documenta **criterios de inclusión/exclusión** (duplicados, severidades); analiza **tendencias internas** más que “valores absolutos” de referencia externa. (ISO/IEC 25023; IEEE-1061).

Escape rate (DER). Porcentaje de **defectos que llegan a producción** respecto del total detectado en el ciclo (pre + post-release).

$$DER(\%) = \frac{\#\ defectos\ en\ producción}{\#\ defectos\ totales} x\ 100$$

Segmenta por **severidad y canal** (Web/API/Móvil) para priorizar riesgos. (Guías prácticas del ecosistema; ejemplos paso a paso).

MTTR (*Mean Time to Restore/Recover*). Tiempo medio para **restaurar** el servicio desde el inicio del impacto. Aclara qué variante usas (recover/repair/resolve/respond) y **marca de inicio/fin**; evita interpretaciones ambiguas y complementa con mediana/p90. (SRE: *Incident Metrics in SRE*; Atlassian, guía de KPIs de incidentes).

Buenas prácticas de gobierno de KPI.

- Publica **definiciones operacionales** en el tablero (fórmulas, ventanas, exclusiones).
- Normaliza **campos de origen** (issue tracker, APM) y automatiza el cálculo (ETL/SQL).
- Acompaña con **SLI/SLO** (p95, error rate) para enlazar calidad del producto con confiabilidad del servicio. (SRE workbook / *Art of SLOs*).

b. Métricas DORA y fiabilidad operativa

Las cuatro claves. DORA identifica cuatro métricas que resumen el desempeño de entrega: ***Deployment Frequency, Lead Time for Changes, Change Failure Rate y Time to Restore Service* (MTTR).** Las dos primeras miden **velocidad**; las dos últimas, **estabilidad**; juntas predicen mejores resultados organizacionales.

Definiciones operativas.

- **Deployment Frequency (DF):** cuántas veces despliegas a producción en una ventana dada.
- **Lead Time for Changes (LT):** tiempo desde **commit** hasta que el cambio está **en producción**.
- **Change Failure Rate (CFR):** % de despliegues que causan ***incidente/rollback***.
- **MTTR (DORA):** tiempo medio para **restaurar el servicio** tras un incidente en producción. (DORA/ Google Cloud y portal DORA).

Recolección de datos (ejemplo práctico). Puedes implementar la tubería **Four Keys** (open source) para recolectar eventos de GitHub/GitLab, procesarlos en **Cloud Run + Pub/Sub** y construir paneles en BigQuery/Looker Studio con las cuatro métricas.

Clasificaciones y tendencias. Los reportes DORA recientes siguen utilizando estas métricas para clasificar equipos (Elite/High/Medium/Low) y analizar mejoras anuales (p. ej., reducción de LT, mayor DF). Utilízalas para **tendencias internas** y **objetivos realistas**, no como comparativas absolutas con otras organizaciones.

Vincular DORA con fiabilidad (SRE).

- **SLI/SLO:** mide disponibilidad y p95/p99 con **histogramas** Prometheus y **exemplars** para saltar a trazas representativas en Grafana.
- ***Error budgets:*** gobierna el *trade-off* de velocidad vs. estabilidad (si consumes presupuesto, reduce el ritmo de cambios).
- ***Apdex*** (opcional) para comunicar satisfacción de usuario (0–1) a perfiles ejecutivos.

c. Construcción de dashboards ejecutivos (Grafana / Power BI)

Principios de diseño (orientados a dirección).

1. **Una página, una historia:** arriba **tarjetas** con KPIs (DF, LT, CFR, MTTR; DER; defect density; SLO cumplimiento), al centro **sparklines** y variaciones contra objetivo, abajo **diagnóstico** (p95, error rate, saturación USE).
2. **Definiciones visibles:** cada panel debe mostrar **fórmula y ventana** (evita malinterpretaciones).
3. **Percentiles, no promedios** para latencia; muestra **p95/p99** y **error budget burn**.
4. **Saltos de contexto:** habilita **exemplars** para ir de una muestra anómala a su **traza** (Jaeger) y logs.

Grafana (con Prometheus).

- **Latencia p95 por ruta** (PromQL):

```
histogram_quantile(0.95,
  sum by (le, route)(rate(http_server_duration_seconds_bucket[5m]))
)
```



(Usa histogramas y etiquetas claras; ver prácticas Prometheus).

- **Disponibilidad (éxitos/total):**

```
sum(rate(http_requests_total{status=~"2..|3.."}[5m]))

/ sum(rate(http_requests_total[5m]))
```

Añade **umbral SLO y burn rate** (reglas multi-ventana).

- **Visualización de histogramas** en Grafana: usa *heatmaps* y *bar gauges* siguiendo la guía oficial.
- **Buenas prácticas de paneles:** tipografías consistentes, colores por estado, uso de *variables* por servicio/entorno.

Power BI (modelo semántico + DAX).

- **Modelo sugerido:** tablas **Deployments, Incidents, Commits, Issues**; relaciones por *repo/app, release, deployment_id.*
- **DAX — CFR (%):**

```
CFR % :=

DIVIDE(

  COUNTROWS(FILTER(Deployments, Deployments[HadIncident]=TRUE())),

  COUNTROWS(Deployments)

)
```

- **DAX — Lead Time (días)** (commit → deploy):

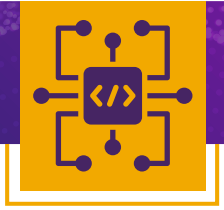
```
LeadTime (days) :=

AVERAGEX(

  Deployments,

  DATEDIFF(Deployments[FirstCommitAt], Deployments[DeployedAt], DAY)

)
```



- **DAX — p95 de latencia** (si cargas muestras o buckets):

```
P95 Latency (ms) :=

PERCENTILEX.INC(

  Requests,

  Requests[DurationMs],

  0.95

)
```

(Funciones *DAX PERCENTILEX.INC / PERCENTILE.INC*).

- **Tarjetas ejecutivas:** coloca **DF, LT, CFR, MTTR** como *cards* y **tendencias** con *sparklines*; usa **tooltips** con definiciones y ventanas. (Buenas prácticas generales de panel).

Guía exprés de instrumentación para el tablero

- **DORA:** adopta **Four Keys** o pipeline equivalente (ETL de PRs, commits, despliegues, incidentes) con reglas de negocio versionadas.
- **QA KPIs:** unifica severidades, *found in* (QA/Prod) y estados "válidos". Sustenta **defect density** con KLOC/FP confiable (mismo método en el tiempo).
- **Confiabilidad:** SLI/SLO con Prometheus (histogramas) y **alertas por burn-rate**; Apdex para público no técnico.

2. Ciclo de vida del defecto y triage

a. Flujo New → Closed (severity/priority)

El ciclo de vida de un defecto define **estados** y **reglas** desde que se reporta (New) hasta su **cierre verificado** (*Closed*). Estandarizarlo mejora la trazabilidad, la priorización del trabajo y la calidad del reporte (ISO/IEC/IEEE 29119-2/3; ISTQB CTFL v4.0.1). (ISO/IEC/IEEE, 2021; ISTQB, 2024).

I. Estados mínimos y transiciones recomendadas

- **New / Open.** Defecto registrado con datos mínimos (título, pasos, resultado esperado/obtenido, *attachments*). (ISTQB, 2024).
- **Triaged.** Equipo de *triage* **valida reproducibilidad, clasifica** (severidad/prioridad), **asigna** responsable y decide siguiente acción (corregir, posponer, rechazar). (ISTQB, 2024; ISO/IEC/IEEE 29119-2).
- **Assigned / In Progress.** Desarrollo implementa la corrección o realiza análisis adicional. (Azure DevOps).
- **Resolved** (con **Resolution**): *Fixed, Duplicate, Won't Fix, By Design/Invalid, Cannot Reproduce, Deferred.* **"Resolved" exige establecer la *resolution*** (p. ej., *Fixed*). (Atlassian; Bugzilla). (Atlassian, 2025; Bugzilla, 2024).
- **Verified.** QA confirma que la corrección **elimina la falla** en la versión/entorno objetivo; si no, **Reopened.** (Bugzilla, 2024).
- **Closed.** Se cumplen criterios de cierre (ver abajo); si el flujo lo requiere, habilita transición para *Reopen*. (Atlassian, 2025).

Equivalencias típicas por herramienta:

Jira: *To Do* → *In Progress* → *Done/Resolved/Closed* (con **Resolution**); **Azure Boards**: *New* → *Active* → *Resolved* → *Closed*; **Bugzilla:** *NEW* → *ASSIGNED* → *RESOLVED* → *VERIFIED* → *CLOSED*. (Microsoft, 2024; Bugzilla; Atlassian).

II. Severity vs Priority (definiciones operativas)

- **Severity (severidad): grado de impacto** del defecto en la operación o desarrollo del sistema. (ISTQB, 2024).
- **Priority (prioridad): importancia de negocio** asignada para atender el ítem (p. ej., orden de corrección). (ISTQB, 2024).

Matriz

- *Severidad:* **Critical** (pérdida de datos/seguridad/indisponibilidad), **Major, Minor, Trivial.**
- *Prioridad:* **P1 (Ahora), P2 (Próximo), P3 (Planificado).**

Nota: **No** confundir términos: un defecto puede ser **severo**, pero de **baja prioridad** (si hay *workaround* temporal), o al revés (ISTQB; IEEE 1044). (ISTQB, 2024; IEEE, 2009).



III. Plantilla de reporte y criterios de calidad

Basado en **ISO/IEC/IEEE 29119-3** (incidente/defecto), un reporte incluye **identificador, resumen, contexto** (objeto/entorno), **pasos de reproducción, resultados esperado/actual, evidencia, severidad, prioridad, estado, referencias** (caso de prueba, *commit*, requisito). (ISO/IEC/IEEE 29119-3; ISTQB, 2024).

Checklist rápido

- ☐ Reproducible y con datos/entorno claro.
- ☐ Impacto (severidad) y urgencia (prioridad) justificados.
- ☐ Evidencia adjunta (logs, *screenshots, trace_id*).
- ☐ Vínculos entre **caso de prueba** y **requisito** (trazabilidad). (ISTQB, 2024; ISO/IEC/IEEE 29119-3).

IV. Políticas de triage y SLA de atención (ejemplo formativo)

- **Asignación:** *Product Owner/QA Lead* decide **prioridad**; *Tech Lead* valida **causa** y **esfuerzo**.
- **SLA sugeridos** (ejemplo didáctico; ajusta a tu contexto):
  - **P1/Critical:** diagnóstico ≤ **2 h**, *hotfix* ≤ **24 h**.
  - **P2/Major:** plan de corrección ≤ **2 días**.
  - **P3/Minor:** agrupar en siguiente *sprint/release*.
- **Reglas:** todo *Resolved* debe llevar **Resolution; Duplicate** enlaza al **bug origen; Deferred** justifica **riesgo/respaldo** (y etiqueta para seguimiento). (Atlassian; Bugzilla; Microsoft).

V. Criterios de entrada y cierre (DoR/DoD) para defectos

- **DoR (para mover a *In Progress*):** reproducible, alcance claro, severidad/prioridad definidas, *attachments* listos.
- **DoD (para *Closed*):** fix **mergeado** y **desplegado** en el **entorno objetivo**; **prueba de confirmación** (*retest*) **OK; regresión** crítica ejecutada; **Resolution** correcta; **notas** de versión actualizadas. (ISTQB, 2024; Bugzilla).

VI. Mapeo a herramientas (buenas prácticas)

- **Jira:** define pantallas de transición para **capturar Resolution** en *Resolved/Closed*; configura transición **Reopen** desde *Closed* si tu proceso lo permite. (Atlassian, 2025).

- **Azure DevOps:** usa **estados** y **categorías** (New/Active/Resolved/Closed) y consulta por **Severity/Priority** para *triage* y reportes. (Microsoft, 2024).

- **Bugzilla:** usa **RESOLVED** con *Fixed/Duplicate/Won't Fix/Invalid* y **VERIFIED** tras QA; al marcar **Duplicate**, enlaza el ***bug*** original. (Bugzilla, 2024; GCC Bugzilla).

VII. Errores frecuentes (y cómo evitarlos)

- **Confundir severidad con prioridad** → documenta definiciones y usa **matriz** en *triage*. (ISTQB, 2024).

- **Cerrar sin verificación** → establece estado **Verified** antes de *Closed* cuando aplique. (Bugzilla).

- **Falta de Resolution** en *Resolved/Closed* → configura pantallas de transición obligatorias. (Atlassian).

- **Duplicados sin enlace** → usa **Resolution=Duplicate** y referencia canónica. (Bugzilla).

b. Roles y dinámica de la reunión de triage

La reunión de *triage* alinea al equipo para **validar, clasificar y decidir** el curso de acción de los defectos nuevos o reabiertos, asegurando un flujo **New → Closed** ágil y trazable. Debe basarse en definiciones operacionales de **severidad** y **prioridad**, y en un proceso de gestión de defectos con estados y transiciones claras (ISO/IEC/IEEE 29119-2/3; ISTQB).

I. Quién participa y con qué rol (RACI del *triage*)

Una forma práctica de clarificar responsabilidades es usar una **matriz RACI** (*Responsible, Accountable, Consulted, Informed*) para cada actividad del *triage*.

Tabla 1. Matriz RACI

Actividad del *triage*	Responsable (R)	Accountable (A)	Consulted (C)	Informed (I)
Preparar listado de New/Reopened, evidencias y duplicados	*QA Lead / Analista QA*	*QA Lead*	*Dev Lead, PO*	Equipo
Confirmar reproducibilidad y alcance	*Analista QA*	*QA Lead*	*Dev/SME*	Equipo

Asignar severidad (impacto técnico)	*QA Lead / Test Lead*	*QA Lead*	*Dev Lead, Arquitecto*	Equipo
Asignar prioridad (urgencia negocio)	*Product Owner (PO)*	*PO*	*QA Lead, Stakeholders*	Equipo
Decidir Resolution (*Fix, Duplicate, Won't Fix, Deferred…*)	*Dev Lead + PO*	*PO*	*QA Lead, Arquitecto*	Equipo
Asignar responsable y fecha objetivo	*Dev Lead*	*Dev Lead*	*PO, QA Lead*	Equipo
Definir prueba de confirmación y regresión mínima	*QA Lead*	*QA Lead*	*Dev Lead*	Equipo
Notificar estado/ seguimiento	*QA/PMO*	*QA/PMO*	-	*Stakeholders*

Fuente: autoría propia.

(Adapta los roles a tu contexto; mantén un "A" único por decisión para evitar ambigüedad.)

Apoyo normativo y de herramientas. La 29119-2 aborda procesos de gestión y control de pruebas/defectos; ISTQB enfatiza reportes claros y criterios de severidad/prioridad; Jira/Azure recomiendan reuniones periódicas y automatización de campos para priorización.

II. Dinámica sugerida (agenda de 30–45 min)

Antes de la reunión (pre-*triage*, 10–15 min):

1. **Consulta guardada** de "New / Reopened" con columnas clave (severidad, prioridad, componente, versión, *attachments, links* a caso/requisito). En Azure Boards existe un ***triage mode*** para revisar y rankear rápidamente.

2. **Higiene del *backlog*:** agrupar sospechas de **duplicado**, adjuntar evidencias mínimas (pasos, *logs, trace_id*), y etiquetar seguridad/accesibilidad si aplica. (ISTQB; 29119-3).

Durante la reunión (30–45 min):

- **Apertura (5 min):** recordar definiciones de **severidad/prioridad** y SLA internos de atención (p. ej., P1 diagnóstico ≤ 2 h). (ISTQB; mejores prácticas de Atlassian/Azure).



- **Revisión rápida por ítem (timebox 2–4 min/defecto):**

  1. ¿Es **reproducible**? Si no → *Cannot Reproduce* o *Needs Info*.
  2. **Clasificación:** severidad (impacto técnico) y prioridad (urgencia de negocio). (ISTQB).
  3. **Decisión:** *Fix* (asignar responsable/fecha), *Duplicate* (enlazar canónico), *Won't Fix/By Design, Deferred* (justificar riesgo), o mantener ***Triaged*** pendiente de información. (Jira/Bugzilla).
  4. **Cierre de la ronda:** definir **prueba de confirmación** (*retest*) y mínima **regresión** asociada (29119-2/3).

- **Cierre (5 min):** revisar **bloqueantes** para el release, acuerdos y responsables (RACI), y métricas de salud del *backlog.* (Azure Boards).

Cadencia. Semanal en desarrollo activo; **diaria** en fases de estabilización o *hardening*. (Azure Boards).

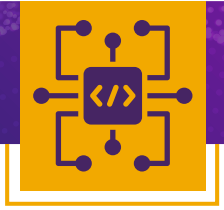
III. Métricas operativas del *triage* (para tu tablero)

- **Time to Triage (TTT):** tiempo medio desde *New* hasta *Triaged/Assigned*.
- **% con Resolution correcto** en *Resolved/Closed* (evita "Done" sin *Resolution*). (Jira).
- **Aging del backlog** por severidad/prioridad.
- **Reopen rate** (% reabiertos tras *fix*).
- **Asignación correcta** (proxy): uso de técnicas de recomendación/ML para sugerir desarrollador adecuado —evidencia de investigación clásica en *bug triage*.

Nota: estas métricas **no sustituyen** KPIs de calidad (DER, *defect density*) sino que miden **higiene de proceso** y ayudan a anticipar escapes. (ISTQB CTAL-TM).

IV. Buenas prácticas (checklist rápido)

1. **Una sola fuente** para reportar bugs (portal/servicio) y **campos obligatorios** (pasos, entorno, evidencia), reduciendo ruido y *back-and-forth*. (Atlassian).
2. **Automatiza la prioridad** cuando sea posible (reglas por impacto/urgencia, colas, usuarios afectados). (Jira Service Management).



3. **Evita mezclar incidentes y defectos:** incidentes operativos usan otra cadencia/roles; conecta, pero no confundas flujos. (Atlassian, guía de severidad de incidentes).

4. **Resuelve duplicados** temprano y **enlaza** el canónico (limpia el backlog y concentra señales). (Bugzilla).

5. **Publica RACI + acuerdos** del *triage* (quién decide qué, y en qué plazos). (PMI/RACI).

V. Antipatrones frecuentes

- **Reuniones sin decisiones** → falta de un A claro (Accountable). Usar RACI y timeboxes.
- **Done sin *Resolution*** → configurar pantallas de transición para exigirla. (Jira).
- **Debatir casos sin evidencia** → requisitos de reporte (29119-3) y gates de calidad del bug.
- **Prioridad = Severidad** (confusión) → recordar definiciones (ISTQB).

c. Gestión de regresiones y hotfixes

Una **regresión** es un defecto introducido o descubierto en áreas no cambiadas tras una modificación; su gestión exige detectar rápidamente el fallo, **reproducirlo**, aislar la causa y **prevenir su reaparición** con una prueba que lo "congele" (*regression test*). Un ***hotfix*** es una corrección urgente aplicada a una versión **en producción** para restaurar el servicio o mitigar impacto severo; debe incluir control de versiones, validación de calidad y despliegue seguro (ISTQB; ISO/IEC/IEEE 29119).

I. Política de regresiones (mínimo viable y repetible)

- **Definir cuándo se activa:** fallas *post-release*, reopens de defectos, errores tras *merge* o cambio de dependencia.
- **Obligatorio el test que falla:** antes del fix, crear (o ajustar) una prueba automatizada que reproduzca la regresión; el fix debe hacerla pasar (ISTQB).
- **Alcance de regresión**, según riesgo: combinar pruebas dirigidas (módulo afectado) + **suite regresión** priorizada por criticidad (ISO/IEC/IEEE 29119).
- **Trazabilidad:** vincular el defecto ↔ commit/PR caso de prueba (id) ↔ *release* para auditoría y métricas (DER, CFR).

Consejo: etiquetar la prueba creada como ***golden regression*** y protegerla frente a cambios (no permitir *quarantine* sin justificativo).



II. Hotfix: flujo operativo recomendado (con variantes de branching)

Detección y estabilización (minutos)

- Declarar **incidente** y **severidad** (P0/P1), activar *runbooks* (rollback, *toggle off, canary*) y restaurar el servicio si procede (Google SRE).

Reproducción y test que falla (horas)

- Reproducir el defecto, aislar el cambio causante y **escribir la prueba** que lo captura (ISTQB).

Estrategia de *branching* (elegir la que usa la organización; en todas exigen **back-merge** para evitar que el fallo regrese):

- ***Gitflow*** (clásico): crear hotfix/x.y.z **desde *main***, corregir, ***merge* a *main* y a *develop/release, tag*** de versión (parche); cerrar rama (Atlassian, Driessen).

- ***Release Flow*** (Microsoft): el fix se hace en ***main*** y luego se ***cherry-pickea*** al **release/x** activo; o al revés si solo aplica a esa versión (DevBlogs; Azure Docs).

- ***Trunk-based*** (escala continua): crear **rama corta** desde *main, fix forward* o revertir, ***merge* inmediato** y cherry-pick al release si existe (AWS Prescriptive Guidance).

Versionado y notas

- Usar **SemVer:** incrementa **PATCH** (x.y.**z+1**) para hotfix; registrar *CHANGELOG* con referencia al defecto y riesgo mitigado (SemVer).

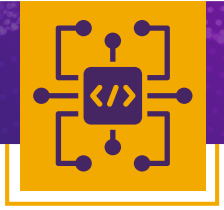
Validación de calidad (antes de producir)

- **Compilar/Probar:** unidad + integración del módulo afectado + **suite de regresión** priorizada + *smoke* end-to-end.

- **Umbrales** en CI/CD (*quality gates*): no permitir despliegue si falla la ***golden regression*** o si excede p95/error-rate definidos (SRE).

Despliegue y verificación segura

- ***Canary o Blue/Green*** con **comparación de SLI** y ***rollback*** automático ante consumo acelerado del ***error budget*** (SRE Workbook).



- ***Feature toggles/flags*** para activar/desactivar el cambio y reducir *blast radius* (Fowler).

Cierre y aprendizaje

- ***Back-merge*** obligatorio (para que el fix quede también en la rama de desarrollo) y ***post-mortem* sin culpa** si el incidente lo amerita (Google SRE).

III. Checklists prácticos

Checklist de hotfix

1. Incidente declarado (severidad, *commander, scribe*) y mitigación temporal si aplica.
2. **Prueba que falla** creada y reproducibilidad documentada.
3. Rama según política (**Gitflow / Release Flow / Trunk-based**) creada con *naming* claro.
4. *Fix* + **pruebas** pasan en CI; cobertura de regresión relevante.
5. **SemVer (PATCH)** aplicado y ***tag*** creado; *release* notes con ID del defecto.
6. ***Canary/Blue-Green*** → monitoreo de SLI y ***rollback*** si *burn* rate alto.
7. ***Back-merge*** a *develop/main o cherry-pick a release* (según modelo).
8. Cerrar defecto con ***Resolution*** correcta y adjuntar evidencia de *retest* + regresión mínima (ver §2.a).

Checklist de regresión (suite)

- Cobertura de **casos críticos** (funcionales y no funcionales pertinentes) priorizados por **riesgo** (29119-1).
- *Golden regressions* con etiquetas inamovibles; prohibido desactivarlas salvo justificación.
- Métricas: ***Reopen rate*** y **tendencia** de fallas repetidas por componente; alimentar el tablero de **CFR/DER**.

IV. Buenas prácticas

- **El *hotfix* no es un atajo:** pasa por **PR** con revisión, *lint, unit/integration* y **regresión mínima**; evita **empujar a producción** sin control (SRE).

- **Deuda de *toggles*:** los *feature flags* son temporales; limpiar cuando el *fix* se consolida para reducir complejidad (Fowler).

- **Estrategia única y publicada:** documenta si usas ***Gitflow, Release Flow*** o ***Trunk-based***, y **cómo** hacer *back-merge/cherry-pick*; los equipos evitan ***doble fix*** o repos divergentes (Atlassian; Microsoft).

- **Despliegues graduales siempre:** *canary/blue-green* con reglas objetivas de éxito (p95, error rate, SLO) y ***rollback*** probado regularmente (SRE Workbook).

V. Antipatrones (y cómo evitarlos)

- **Arreglar sólo en la rama de *release*** (olvidar *back-merge*) → el *bug* **reaparece** en la siguiente versión. **Solución:** política de ***back-merge*** obligatoria (Azure Docs).

- **Hotfix sin prueba de regresión** → el fallo vuelve con otro cambio. **Solución:** *golden regression* obligatoria (ISTQB).

- **Versionado inconsistente** → usuarios no saben qué *release* contiene el *fix*. **Solución: *SemVer*** y *tagging* consistente (SemVer).

- **Despliegue *big bang*** de emergencia → alto riesgo. **Solución:** *canary/blue-green y feature flags* (SRE; Fowler).

3. Análisis causa-raíz

a. 5 Whys, Diagrama Ishikawa, Pareto

Estas tres técnicas se complementan: **Ishikawa** abre el abanico de causas posibles (divergir), **Pareto** prioriza con datos las causas **vitales** (enfocar), y **5 Whys** profundiza hasta llegar a causas raíz accionables (convergir). Usarlas en ese orden evita *túnel* y garantizar soluciones medibles. (ASQ, s. f.; Alkiayat, 2021).

I. 5 Whys (cinco porqués)

¿Qué es?
Técnica iterativa que pregunta **¿por qué?** sucesivas veces sobre el efecto observado para descubrir causas subyacentes hasta una hipótesis de raíz y su contramedida. Se originó en Toyota y se usa en mejora continua (*Kaizen*). (ASQ, s. f.).

¿Cuándo conviene?
Problemas **acotados** y **bien definidos**, donde el flujo causal es claro y la evidencia está disponible; se puede combinar con Ishikawa (ramificar) y **5 Hows** para detallar soluciones. (ASQ, s. f.).

Pasos guiados.

1. Enunciar el **problema observable** (hecho verificable, no juicio).
2. Reunir **evidencia** (logs, métricas, *trace_id*, capturas).
3. Aplicar la cadena de **porqués** (3–7 iteraciones), **validando** cada respuesta con datos.
4. Identificar la **causa controlable** más profunda y definir **contramedida + dueño + fecha.**
5. Registrar el hilo causa-efecto (tabla) y vincular con el defecto/caso. (IHI; ASQ).

Limitaciones (y cómo mitigarlas).
5 Whys **puede** parar en síntomas, promover **una sola** causa y depender de la pericia del facilitador; distintos equipos pueden llegar a conclusiones diferentes. **Mitiga** triangulando con Ishikawa, verificando con datos y no fijando el "5" como tope rígido. (Card, 2017; Wikipedia, 2025).

Plantilla rápida (tabla 5 Whys)

Problema: ___ / Evidencia: ___

¿Por qué? → ___ (dato: ___)

¿Por qué? → ___ (dato: ___)

¿Por qué? → ___ … → **Contramedida:** ___ / **Dueño:** ___ / **Fecha:** ___

II. Diagrama Ishikawa (espina de pescado / causa-efecto)

¿Qué es?
Es una herramienta visual utilizada para **generar y organizar** las causas de un problema en diferentes **categorías** principales (las **"espinas"**) y sus subcausas. En manufactura suelen emplearse las **6M** (método, máquina, mano de obra, materiales, medición, medio ambiente), mientras que en servicios o TI se adaptan a categorías como **proceso, personas, plataforma/infraestructura, datos, políticas y entorno** (ASQ, s. f.; Ishikawa, 1990).

¿Cuándo conviene?
Al inicio del análisis, para evitar fijar soluciones prematuras y **hacer visible la multicausalidad**; útil en sesiones de *brainstorming* con roles diversos. (ASQ, s. f.; Kumah, 2024).

Pasos guiados (aula/lab).

1. Dibuja **cabeza** con el **efecto** (p. ej., "p99 de /checkout > 2 s").

2. Traza el **espinazo** y agrega 6–8 **categorías** (ajustadas a software).

3. Llena subcausas con hechos (métricas/logs/trazas), no suposiciones.

4. Marca causas controlables e hipótesis a validar.

5. Conecta con Pareto (cuantificar) y 5 Whys (profundizar en las vitales). (ASQ, s. f.; OPM, 2019).

Tip: invita a **ramificar** (no una sola línea). Si hay varias hipótesis, considera **múltiples** Ishikawa más específicos en lugar de uno sobrecargado. (ASQ, s. f.).

III. Pareto (80/20) y gráfico de Pareto

¿Qué es?

Es un principio que plantea que **una minoría** de causas concentra la **mayor parte** del efecto. A partir de esta idea, el diagrama de Pareto organiza las categorías por su **frecuencia o impacto** y muestra la contribución **acumulada**, facilitando la identificación de las áreas prioritarias de acción (Juran Institute, 2019; ASQ, s. f.).

¿Cuándo conviene?

Conviene utilizarlo después de identificar las posibles causas siguiendo un análisis como el de Ishikawa para **priorizar con datos** cuáles abordar primero. También es útil en la fase de control, para **verificar** si las intervenciones aplicadas están reduciendo el efecto en las categorías seleccionadas (Alkiayat, 2021; CEC NSW, s. f.).

Pasos guiados

1. **Define categorías** mutuamente excluyentes (p. ej., **timeout API, índice DB, GC, red**).

2. Cuenta **frecuencia** o estima **impacto** (tiempo/$$/usuarios).

3. Ordena **descendente** y calcula % **acumulado**; traza barras + curva.

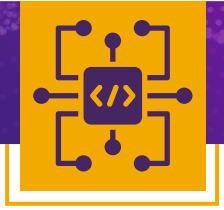
4. Elige el **corte** (p. ej., 70–80 %) y documenta hipótesis de intervención. (ASQ, s. f.).

Precaución: si la muestra es pequeña o la categorización es inconsistente, el **80/20** puede ser ilusorio; normaliza por **exposición** (p. ej., transacciones por ruta). (Alkiayat, 2021).

Flujo integrado recomendado (para QA/operaciones)

1. **Ishikawa** para mapear **todas** las causas plausibles (con evidencia).

2. **Pareto** para seleccionar las **vitales** (por frecuencia o impacto).



3. **5 Whys** sobre cada causa vital para llegar a la **raíz accionable** y definir contramedidas con dueño y fecha.

4. **Verificar** resultados con el mismo Pareto (se espera caída visible en las categorías intervenidas).

Ejemplo

- **Efecto:** p99 de /checkout > 2 s (SLO violado).

- **Ishikawa (resumen):** *Plataforma* → **pool DB saturado**; *Proceso* → despliegues sin *warm-up;* *Datos* → picos con cupones; *Personas* → *runbook* desactualizado; *Medición* → falta *exemplar* para enlazar trazas de pagos. (ASQ, s. f.).

- **Pareto (30 incidentes/semana):** 12 **pool DB**, 8 **timeouts pagos**, 5 GC, 3 **red**, 2 **otros** → **80 %** acumulado en dos categorías. (ASQ, s. f.).

- **5 Whys sobre "pool DB":**

1. ¿Por qué se agota el *pool*? → demasiadas consultas simultáneas.

2. ¿Por qué tantas consultas? → faltan **índices** correctos en tabla *orders*.

3. ¿Por qué faltan índices? → cambio de esquema no actualizó migración.

4. ¿Por qué no se detectó? → **suite de regresión** sin *query-plan check*.

5. ¿Por qué no hay check? → política de revisión no incluye **linterna SQL**.

→ **Contramedidas:** índice + migración; gate de plan de ejecución en CI; ampliar *pool y warm-up* post-deploy. (IHI/ASQ).

Errores comunes (y cómo evitarlos)

- **Quedarse con una sola causa** (sesgo de confirmación). → **Ishikawa** para abrir y 5 Whys **sobre varias ramas.** (Card, 2017; ASQ, s. f.).

- **Pareto con categorías porosas.** → Definir bien y **normalizar** por exposición/impacto. (Alkiayat, 2021).

- **Cinco rígido.** → Usar tantas iteraciones como soporte la evidencia; validar con métricas/logs. (Card, 2017).

b. Técnicas de blameless post-mortem

Un *post-mortem* **sin culpa** busca aprender de incidentes/regresiones sin señalar a personas, examinando **sistemas, condiciones y decisiones** con evidencia (métricas, *logs*, trazas) y generando **acciones de**



mejora verificables. La cultura *blameless* aumenta la fiabilidad al favorecer reportes honestos y completos, condición necesaria para mejorar (SRE Book; Atlassian; Etsy).

I. Principios de un *post-mortem* sin culpa

1. **Aprendizaje > castigo. El costo del fallo es educación:** se privilegia comprender **cómo** ocurrió, no **quién** se equivocó (SRE Book).

2. **Just Culture.** Se separa **acto y contexto:** responsabilidad sí, **culpa no**; la seguridad y el aprendizaje necesitan relatos francos (Dekker).

3. **Seguridad psicológica.** En la invitación y apertura, declara explícitamente el carácter *blameless* (Atlassian).

4. **Hechos verificables.** Reconstrucción **cronológica** con fuentes (APM, *logs, trace_id*) y citas de quienes actuaron, sin juicios (Etsy).

5. **Lenguaje inclusivo.** Evitar culpó/rompió → usar el sistema no protegió contra…/los controles no detectaron…. (Atlassian).

6. **Acciones SMART y seguimiento.** Sin acciones, el documento no genera cambio (SRE Workbook).

***Prime Directive* (Norm Kerth)** para abrir la sesión: *"Independientemente de lo que descubramos, entendemos y creemos que todos hicieron el mejor trabajo posible con lo que sabían en ese momento, sus habilidades, recursos y la situación."* (Kerth, s.f).

II. Estructura mínima del documento (plantilla sugerida)

1. **Resumen ejecutivo** (1–3 líneas), **impacto** (usuarios, duración, SLO afectado).

2. **Detección** (cómo/qué alertó) y **cronología UTC** (minuto a minuto, con enlaces a paneles/trazas).

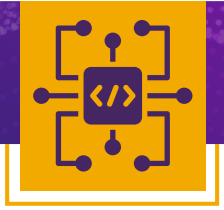
3. **Contribuyentes** (técnicos/organizacionales), **qué funcionó/qué no y dónde tuvimos suerte.**

4. **Causas** (*RCA* con 5 Whys, Ishikawa u otra técnica—evitar detenerse en el primer **por qué**).

5. **Acciones** (*Correctivas/Preventivas*) **SMART, dueño** y **fecha; riesgo residual.**

6. **Evidencia** (gráficas, *logs, trace_id*, PRs) y **glosario.**

(Guías SRE/Atlassian).



III. Dinámica de la reunión (facilitación práctica, 45–60 min)

- **Antes:** enviar **invitación *blameless*** con propósito, *Prime Directive*, enlaces a evidencia y **borrador del *timeline*** (Atlassian).
- **Apertura (5 min):** recordar *blameless*, objetivos y reglas (una conversación, sin interrupciones).
- **Reconstrucción (20–30 min):** cocrear el ***timeline***; el facilitador usa **entrevista cognitiva** ligera (¿qué observaste/esperabas?) para extraer **modelos mentales** (Etsy).
- **Análisis (10–15 min):** agrupar **contribuyentes** (técnicos/proceso/herramientas/organización) y aplicar **5 Whys/Ishikawa** a 2–3 ramas **vitales** (ver §3.a).
- **Acciones (10–15 min):** definir **3–7 acciones** SMART (no solo capacitar/tener cuidado), con **propietario y fecha**; priorizar por **riesgo y costo.**
- **Cierre (5 min):** acuerdos, fecha de **revisión de acciones** y difusión (SRE Workbook; Atlassian).

Tip: Si surge lenguaje de culpa, el facilitador reencuadra a **condiciones del sistema** (¿qué señales/controles/guardarraíles faltaron?).

IV. Ejemplos de acciones eficaces (evita los antipatrones)

- **Eficaces:** *"Agregar verificación de esquema en CI (Job 'db-schema-gate') y alerta de drift; canary 5% con rollback automático si burn-rate > 6×/30m"*. (SRE Workbook).
- **Antipatrones: Tener más cuidado, Capacitar al equipo** (sin contenido), **Despedir al culpable**. No cambian el sistema ni previenen recurrencia (Atlassian; Dekker).

V. Métricas de efectividad del *post-mortem*

- **% de acciones cerradas a tiempo y tiempo mediano a cerrar acciones.**
- **Reincidencia** (mismo patrón de fallo en 30–90 días) ↓.
- **Cobertura de *runbooks* y *guardrails*** creada (p. ej., feature flags, canary, reglas de alerta).
- **Participación:** relatos recogidos de todas las funciones; **diversidad de fuentes** (trazas, *logs*, PRs).

(SRE Workbook; Atlassian).



VI. Integración con cultura y procesos

- **Publicar y compartir** *post-mortems* internamente (*fearless sharing*) para maximizar el aprendizaje organizacional (SRE).

- **Vincular** con **gestión de defectos** (Resolution, regresión que congela el error) y con **SLO/SLA** (lecciones → nuevos *guardrails*). (SRE Book; Atlassian).

4. Integración QA en CI/CD

a. Quality Gates en Jenkins/GitLab

Un *quality gate* es un **conjunto de condiciones objetivas** que una *branch* o *merge request* debe cumplir **antes** de avanzar a release. Suele combinar: **calidad estática** (issues, *code smells, coverage*), **pruebas** (unitarias/integración), **seguridad** (SAST/dependencias) y, si aplica, **criterios de desempeño** mínimos. En SonarQube, el *quality gate* responde explícitamente ¿está listo para liberar?, con condiciones como *coverage en código nuevo* ≥ *80 %*; 0 issues críticos; ese veredicto se puede **bloquear** en CI/CD.

I. Jenkins: *quality gates* en la *pipeline*

a) Estructura base. En Jenkins Pipeline (declarativa o *scripted*) defines stages para ***build*** → ***test*** → **análisis estático** → **gate** y usas pasos nativos/plug-ins.

```
// Jenkinsfile (declarativo)

pipeline {

  agent any

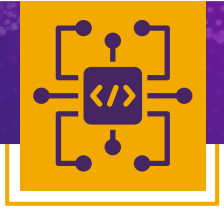
  stages {

    stage('Build') { steps { sh 'mvn -B -DskipTests package' } }

    stage('Unit tests + coverage') { steps { sh 'mvn -B test jacoco:report' } }

    stage('Static analysis (SonarQube)') {

      steps {
```



```
        withSonarQubeEnv('SonarQubeServer') {
          sh 'mvn -B verify sonar:sonar'
        }
      }
    }
    stage('Quality Gate') {
      steps {
        // Bloquea la pipeline si el gate NO es verde
        timeout(time: 5, unit: 'MINUTES') {
          waitForQualityGate abortPipeline: true
        }
      }
    }
  }
}
```

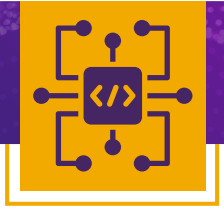
- WaitForQualityGate **pausa** la *pipeline* hasta que Sonar calcule el gate y **falla** si no es "verde". Requiere configurar el *webhook* de Sonar hacia Jenkins.
- Define las **condiciones** del gate (p. ej., *No new critical issues, Coverage on New Code ≥ 80 %*) en SonarQube.

b) Hacer gate de cobertura desde el *build.* Complementa el veredicto con **reglas de JaCoCo** que **fallen** el *build* si la cobertura está **bajo umbral** (p. ej., 80 %) usando el *goal* jacoco:check (lifecycle verify).



```
<!-- pom.xml (extracto) -->
<plugin>
 <groupId>org.jacoco</groupId><artifactId>jacoco-maven-plugin</artifactId>
 <version>0.8.10</version>
 <executions>
  <execution>
   <goals><goal>check</goal></goals>
   <configuration>
    <rules>
     <rule><element>BUNDLE</element>
      <limits>
<limit><counter>LINE</counter><value>COVEREDRATIO</value><minimum>0.80</minimum></limit>
      </limits>
     </rule>
    </rules>
   </configuration>
  </execution>
 </executions>
</plugin>
```

Consejo: publica **JUnit** en Jenkins y evita enmascarar fallos como *UNSTABLE* si tu política exige *FAIL.* (Jenkins JUnit/xUnit).



II. GitLab: quality gates en merge requests

En *GitLab* no existe un *quality gate* único como término oficial, pero se **implementa** combinando **jobs que fallan** ante incumplimientos + **reglas de aprobación/estatus** que **impiden el *merge*** mientras no cumpla.

a) Código y mantenibilidad (Code Quality).

- GitLab muestra *findings* de **Code Quality** en la MR a partir de un **artifact** codequality (formato CodeClimate). La plantilla **Code-Quality.gitlab-ci.yml** existe, pero está **deprecada** (17.3); se recomienda integrar herramientas soportadas directamente o generar el JSON.

```
# .gitlab-ci.yml (ejemplo minimalista)

code_quality:

  stage: test

  image: registry.gitlab.com/gitlab-org/ci-cd/codequality:latest

  script:

    - /codequality -f json -o gl-code-quality-report.json

  artifacts:

    reports:

      codequality: gl-code-quality-report.json

    expire_in: 1 week
```

(Si usas la imagen oficial del escáner; valida compatibilidad con tu lenguaje/proyecto.)

b) Cobertura de pruebas como gate.

Publica **Cobertura/JaCoCo** como artifacts:reports:coverage_report y **muestra cobertura** en la MR; para **gating**, añade un job que **calcule** cobertura y **falle** bajo umbral, o usa políticas de *approvals* que exijan ese job en verde.

```
coverage_gate:
  stage: test
  image: eclipse-temurin:17
  script:
    - mvn -B -q test jacoco:report jacoco:check  # falla si < 80%
  artifacts:
    reports:
      junit: target/surefire-reports/*.xml
      coverage_report:
        coverage_format: cobertura
        path: target/site/jacoco/jacoco.xml
```

c) Seguridad (SAST/Dependencias) como gate.

- Activa **SAST** incluyendo la plantilla oficial; el job genera artifacts:reports:sast y la MR mostrará vulnerabilidades. Puedes **requerir** que la MR tenga **0 críticas** antes de aprobar.

```
include:
  - template: Jobs/SAST.gitlab-ci.yml   # habilita SAST “out-of-the-box”
```

d) Bloquear el *merge* hasta que pase el *quality gate* externo (SonarQube).

- Opción 1 (**simple**): hacer que el job que ejecuta sonar-scanner **espere/consulte** el *quality gate* y **falle** si es rojo.

- Opción 2 (**recomendada en Ultimate**): usar **External Status Checks** para **bloquear la MR** hasta recibir “pass” desde SonarQube (no requiere rejecutar CI si cambia el gate).



III. Diseñar tu *quality gate* (*checklist* recomendado)

1. **Definiciones visibles** en el tablero: fórmula, ventana y umbrales (ej., *coverage new code* ≥ *80 %; 0 Critical; 0 High SAST*).

2. **Condiciones mínimas (no-go):**

   - **Build + Unit tests** sin fallos (JUnit);
   - **Coverage gate** ≥ umbral (JaCoCo *check*);
   - **Análisis estático** (*quality gate* Sonar verde);
   - **SAST / dependency scanning** sin **Críticas**;
   - (Opcional) **Smoke perf** p95 < X ms para 1–2 rutas clave.

3. **Bloqueo de *merge*:**

   - Jenkins: el *stage* Quality Gate **falla** la pipeline.
   - GitLab: **jobs en rojo** + **Approval Rules / Status Checks** que impiden *merge*.

4. **Ejecución temprana (*shift-left*):** corre análisis en MR y muestra hallazgos **en contexto** (widget de MR en GitLab).

IV. Antipatrones (y cómo evitarlos)

- **Gate solo por promedios de cobertura.** Mejor ***coverage* en código nuevo** (evitar **herencia histórica**) y usar **reglas por paquete/módulo.** (Sonar*Qube New Code*; JaCoCo check).
- **Plantillas obsoletas sin mantenimiento.** El *template* Code-Quality está **deprecado** desde 17.3; integrar herramientas soportadas o genera el JSON tú mismo.
- **MR mergeable con gate rojo.** En GitLab, configurar **Approval Rules** y/o **External Status Checks**; en Jenkins, **falla** explícitamente con waitForQualityGate.

b. Política de fail-fast y *feature toggles*

Una política **fail-fast** en CI/CD busca **detectar y cortar** rápido cualquier condición de fallo para ahorrar ciclos (tiempo, dinero) y proteger la calidad. Los **feature toggles** (o feature flags) desacoplan **despliegue**

de **exposición**, permitiendo *rollouts* graduales, *canary y "kill switches"* sin nuevos *deploys*. Juntos habilitan **entrega continua segura** y decisiones basadas en **SLO/error budget.**

I. ¿Qué entendemos por *fail-fast*?

- **En la *pipeline*:** estructura de **etapas tempranas y baratas** (compilar, *lint*, unitarias, SAST) que **frenan** el resto si fallan; paralelización con **fail-fast** para abortar ramas hermanas; **auto-cancel** de *pipelines* obsoletas. (Jenkins/GitLab).

- **En ejecución:** *timeouts* cortos, **circuit breakers** y *health checks* agresivos, para fallar temprano y recuperar rápido (evitar cascadas). (Nygard; Fowler).

- **En gobierno:** actuar según **SLO y error budget:** si el consumo (*burn rate*) se dispara, **parar o revertir** el cambio. (SRE Book/Workbook).

fail-fast = **feedback más temprano** (CD). Organizar el *pipeline* para que lo **barato** falle primero y evite ejecutar lo **caro** (e2e, perf).

II. ¿Qué son los feature toggles y por qué importan?

- Son **interruptores de ejecución** que cambian el comportamiento ***sin redeploy***.

  Tipos útiles: ***Release toggles*** (exposición gradual), ***Ops toggles*** (apagar componentes), ***Experiment toggles*** (A/B), ***Permission toggles*** (cohortes/roles). Los *toggles* **agregan complejidad** y deuda, por lo que requieren gobierno activo. (Fowler).

- En plataformas: **GitLab** y **Unleash** proveen *feature flags* con *targeting* por entorno/segmento y buenas prácticas como **desactivado por defecto**.

- Son el **complemento natural** de **canary**: desplegar a pocos y **encender** paulatinamente ayuda a **fallar rápido** con bajo *blast radius*. (SRE Workbook).

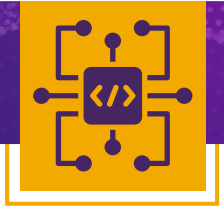
III. Implementación práctica

Fail-fast en Jenkins y GitLab

Jenkins (declarativo) — activar *fail-fast* en paralelos:

```
pipeline {

  agent any
```



```
  options { parallelsAlwaysFailFast() } // aborta ramas paralelas al primer fallo
  stages {
    stage('Lint & Unit') { steps { sh 'mvn -B -q test' } }
    stage('SAST & Analysis') { steps { /* Sonar/SAST */ } }
    stage('Deploy Canary') { when { expression { currentBuild.result == null } } steps { /* ... */ } }
  }
}
```

Tip: Usa timeout en etapas y corta al primer *gate* rojo.

GitLab CI — diseño por **etapas cortas** y ***auto-cancel*** de *pipelines* previas; *fail fast testing* ejecuta un **subset** representativo y **no pasa** a la suite completa si falla:

```
stages: [precheck, test, secure, deploy]
precheck:lints: { stage: precheck, script: ['npm ci','npm run lint'] }
test:subset:
  stage: test
  script: ['npm run test:subset']   # 10% crítico → si falla, no corre la suite completa
test:full:
  stage: test
  needs: ['test:subset']
  rules: [ { if: '$CI_JOB_STATUS == "success"' } ]
  script: ['npm run test']          # corre sólo si pasó el subset
```



Activar en ajustes **Auto-cancel redundant pipelines** y considerar interruptible: true para cancelar *pipelines* anteriores.

***Feature toggles* (patrones mínimos)**

- **Interruptor por entorno** (simple y coste cero):

```
import os

FEATURE_CHECKOUT_V2 = os.getenv("FEATURE_CHECKOUT_V2", "false").lower() == "true"

def handler(req):

    if FEATURE_CHECKOUT_V2:

        return checkout_v2(req)    # ruta nueva (canary/cohorte)

    return checkout_v1(req)        # ruta estable
```

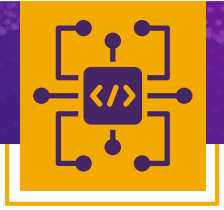
Publicar **métricas y *logs*** con el estado del *flag* para comparar SLI entre rutas. (*SRE Workbook*).

- **Gestor de *flags*** (segmentos y *kill switch*): con **Unleash**/*GitLab Feature Flags* definir un *flag* con **estrategias** (por entorno/porcentaje/cohorte) y cambiar su estado **en caliente** (sin redeploy).
- ***Kill switch:*** *flag* operacional para **apagar** una integración/llamada externa ante errores elevados o *burn rate* alto, mientras CI prepara el *rollback.* (SRE; Nygard—*circuit breaker*).

IV. Política integrada

Objetivo: minimizar **MTTD/MTTR** y **reducir riesgo** de cambios.

1. **Fail-fast en *pipeline***
   - Ordenar etapas por **costo** y **valor de detección**: *lint/unit* → SAST/Analítica → integración → e2e/perf.
   - *Gates* obligatorios (rojo = parar): **build, unit, análisis estático** y **seguridad.** (CD, SonarQube).
   - Paralelos con **fail-fast; auto-cancel** de *pipelines* anteriores en MRs. (**Jenkins**, GitLab).
2. **Uso de *toggles***
   - **Default OFF** al crear *flags* nuevos; solo ***exponer*** tras pasar *gates*. (GitLab *dev docs*).



- **Rollout progresivo:** 1% → 5% → 25% → 100% con **canary** y *observabilidad* por cohorte. (SRE Workbook).
- **Propiedad y TTL:** cada *flag* tiene **dueño** y **fecha de retiro** (*toggle cleanup*) para evitar **deuda de configuración.** (Fowler).
- **Telemetría:** registra feature_flag, variant, user_cohort en métricas/logs para comparar **p95, tasa de error y conversión** entre ramas. (SRE).

3. **Gobierno con SLO/error-budget**

- Si el *burn rate* supera umbral, **pausa** el rollout o **apaga** con *kill switch*; si es estable, **incrementa** exposición. (SRE Workbook).

V. Antipatrones frecuentes (y su corrección)

- **"Flag eterno"** (nunca se retira) → establece **TTL** y *cleanup* en cada release. (Fowler).
- **Demasiados *flags* anidados** → simplifica; evita dependencias entre *flags* y nidos profundos. (Unleash—*best practices*).
- **Fail-fast sólo en teoría** → confirmar que la *pipeline* **para** al primer *gate* rojo y que *pipelines* previas se **auto-cancelan**. (GitLab/Jenkins).
- **Sin *kill switch*** → añadir *ops toggles* y **circuit breakers** para aislar fallos. (Fowler; Nygard).

VI. Guía de despliegue seguro con *flags* (paso a paso)

1. *Merge* a main luego *gates* verdes → despliegue automático **con flag OFF**.
2. Activar **1%** (cohorte interna) → verificar **SLI** (p95, error rate) vs. control.
3. Aumentar a **5–25%** → **SLO burn-rate** bajo → seguir; alto → **OFF** y *rollback*.
4. Llegar a **100%** y **retirar el flag** (borrar código muerto). (SRE/Fowler).

5. Mejora continua y reporting

a. Kaizen & retros del equipo QA

La mejora continua en QA combina una **cultura Kaizen** (pequeños cambios frecuentes guiados por PDSA/PDCA) con **retrospectivas** estructuradas por iteración para transformar hallazgos en **acciones medibles.** En Scrum, la **Sprint Retrospective** existe precisamente para "planificar formas de aumentar la calidad y la eficacia" del equipo; por su parte, **Evidence-Based Management (EBM)** propone medir resultados y experimentar con hipótesis de mejora.

I. Fundamentos de Kaizen aplicados a QA

- **Definición y enfoque:** *Kaizen* = mejorar **día a día** mediante ajustes pequeños y sostenidos, desde el gemba (lugar del trabajo), con participación de todo el equipo. (Imai, 1986).
- **Ciclo PDSA/PDCA:** *Plan–Do–Study/Check–Act* para iterar sobre hipótesis y estandarizar lo que funciona. (Deming Institute).
- **A3 de mejora:** formato de una página que guía problema–análisis–contramedidas–seguimiento, útil para consolidar *retros* en **acciones trazables.** (MIT Sloan; Lean.org).
- **Toyota Kata:** rutina de mejora (*Improvement Kata*) y de *coaching* para practicar objetivos, condición objetivo, experimentos y aprendizaje—muy alineado con micromejoras en QA. (McGraw-Hill).

II. Retrospectivas (estructura y dinámica)

- **Estructura en 5 pasos** (Derby & Larsen): *(1) Abrir el espacio, (2) Recoger datos, (3) Generar insights, (4) Decidir acciones, (5) Cerrar.* Utilizar tanto en *sprints* funcionales como en ciclos de **pruebas.**
- **Marco cultural:** inicia con la ***Prime Directive*** de Norm Kerth para fomentar seguridad psicológica (todas las personas hicieron lo mejor que pudieron con lo que sabían…).
- **Agenda sugerida (45–60 min)**

  1. *Set the stage* (5'): objetivo y acuerdos (*blameless*).
  2. *Gather data* (10–15'): p. ej., **DORA** (DF, LT, CFR, MTTR), **QA-KPIs** (DER, defect density), p95/p99, escapes por severidad.
  3. *Generate insights* (10–15'): 5 Whys/Ishikawa sobre 1–2 problemas vitales.

4. *Decide what to do* (10–15'): **3–5 acciones SMART** con dueño/fecha; registrar en **A3.**

5. *Close* (5'): *ROTI* y compromisos. (Scrum Guide + EBM + técnicas de retrospectiva).

III. ¿Cuándo usar qué ?(Kaizen, Retro, A3, Kata)

Tabla 2. Uso de Kaizen, Retro, A3 y Kata

Situación	Herramienta principal	Resultado esperado
Mejora operativa simple (p. ej., reducir *flakiness* de un test)	**Kaizen + PDSA**	Cambio pequeño estandarizado
Cierre de iteración / *sprint*	**Retro** (5 pasos)	Lista de **acciones SMART** priorizadas
Problema recurrente con varios factores	**A3** (causa-raíz + plan)	Documento 1 página con seguimiento
Desarrollo de hábito de mejora	**Toyota Kata** (rutina)	Ciclo de práctica y *coaching*

Fuente: autoría propia.

IV. Tablero Kaizen (operativizar la mejora)

- **Columnas:** *Idea* → *Aprobada* → *En experimento (PDSA)* → *Estandarizada* → *Medida (EBM).*
- **Políticas:** WIP bajo; cada tarjeta con **métrica objetivo** (p. ej., "↓CFR de 15% → 10%" o "↑cobertura en *new code* a 85%"). (EBM: *outcomes over outputs*).

V. Métricas de efectividad (mejora y retro)

- **% acciones cerradas** y ***lead time*** para cerrar acciones de retro.
- **Impacto** en ***outcomes*** (EBM): *Customer Satisfaction, Time-to-Market, Ability to Innovate, Current Value.*
- **Tendencia** en **DER/CFR/MTTR** tras 2–3 iteraciones (evitar conclusiones por 1 sola). (Scrum/EBM).

VI. Buenas prácticas (*checklist*)

1. **Una acción por persona** como máximo y con **fecha clara** (evita listas aspiracionales). (Derby & Larsen).



2. **Evidencia:** enlaza acciones a **gráficas** o **consultas** (Prometheus/Grafana, defectos, *lead time*). (Scrum EBM).

3. **Cierra el bucle:** revisa acciones **al inicio** de la siguiente retro. (Scrum Guide).

4. **Estandariza** lo que funcionó (último paso del PDSA) y **documenta** en tu *playbook*. (Deming).

5. **Haz visible la mejora:** usa **A3** y un ***Kaizen board***; celebra logros pequeños -cultura Kaizen-. (Imai; Lean.org).

VII. Antipatrones (y cómo evitarlos)

- Retro = terapia de grupo **sin datos** → traer **métricas** (EBM/DORA/QA-KPIs) y evidencias (logs/trazas).
- Solo brainstorming, **sin acciones SMART** → fijar **máximo 3–5** y colocar en el **Kaizen board.** (Derby & Larsen).
- Culpa personal → recordar ***Prime Directive***, reencuadra en **sistemas/controles**.
- Mejoras no estandarizadas → aplicar **PDSA** hasta *Act* y documentar el nuevo estándar. (Deming).

b. Comunicación de hallazgos a stakeholders

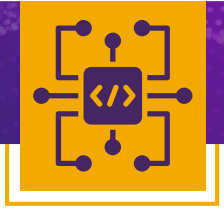
Convertir resultados técnicos (KPIs de QA, métricas DORA, SLO/SLA, evidencias de pruebas) en **mensajes accionables** para audiencias diversas. La regla de oro: **BLUF** (*bottom line up front*)—empieza por la conclusión y la recomendación, y luego respáldala con datos y anexos. Este enfoque—conocido también como **Minto Pyramid Principle**—mejora la comprensión ejecutiva bajo presión de tiempo.

I. ¿Qué debe entregarse y con qué formato?

Artefactos mínimos estandarizados (ISO/IEC/IEEE 29119-3):

- ***Test Status Report*** (avance en curso, riesgos, *blocking issues*).
- ***Test Completion Report*** (resultado final por objetivo, cobertura alcanzada, incidentes críticos, lecciones).
- ***Test Incident Report*** (defectos/incidentes con impacto).

  La norma incluye **plantillas y campos obligatorios** para estos reportes, útiles como **contratos** de comunicación en QA.



Para incidentes y hotfixes: adoptar la estructura de **comunicaciones de incidentes** del SRE Workbook/ Guide (rol de *Communications Lead*, cadencia, lista de distribución), de modo que **quien arregla** no sea **quien comunica**, evitando *context-switch* costoso.

Principios de redacción: lenguaje claro (*Plain Language*) con títulos descriptivos, frases activas y glosario; evita jerga no necesaria.

II. Mapear *stakeholders* y mensajes (quién necesita qué)

Usar una **matriz RACI** como base del **plan de comunicaciones:**

- **Ejecutivos/negocio (A/I):** decisión ***go/no-go***, riesgos y costo/beneficio. Piden **resumen** (1 página), **tendencias** y variación vs. objetivo.
- **Producto/PO (A/C):** impacto en *roadmap* y usuarios, **DER/CFR**, *escape rate.*
- **Arquitectura/Dev (R/C):** defectos críticos, *root causes, capacity y tech debt* priorizada.
- **Operaciones/SRE (R/C):** SLI/SLO, **burn rate**, alertas y *runbooks*.
- **Cumplimiento/cliente (I):** trazabilidad, evidencias, acuerdos de nivel de servicio.

  Una RACI bien definida **evita ambigüedades** y sirve de línea base para frecuencia/canal y nivel de detalle.

III. Estructura de un mensaje efectivo (guion de 6 pasos)

1. **BLUF (1–3 líneas):** decisión o recomendación ("**Go con reserva** si DER<2% y p95 < 300 ms en *checkout*; abrir *tech debt* para *pool* de DB").
2. **Contexto breve:** qué probamos, ventana, ambiente y *scope* (norma 29119-3).
3. **Evidencia clave (gráficas):** p95/p99, *error rate*, cumplimiento SLO/SLA, DER/CFR/MTTR—**percentiles, no promedios.**
4. **Riesgos y mitigaciones:** tabla RAG con criterios definidos (no colores **a ojo**).
5. **Acciones y responsables (RACI):** 3–5 acciones *SMART* con dueños y fechas.
6. **Anexos técnicos:** *logs/trazas*, consultas, casos de prueba, *post-mortem* (cultura *blameless*).

Plantilla de asunto: [QA Readout][Release vX.Y] BLUF: GO/NO-GO + 3 riesgos (p95, DER, CFR).

IV. Visualización y dashboards para *stakeholders*

Cómo contar la historia con datos (HBR + NN/g):

- Elegir el **tipo de gráfico** según la **tarea** (comparar, distribuir, relacionar) y evita decorativos; **anota** la línea de SLO/SLA y el objetivo.
- Para **tableros ejecutivos**, usar panel **de una página** que permita comprensión **de un vistazo** (preattentive features, sparklines, tarjetas KPI).
- Incluir **definición visible** de cada métrica (fórmula/ventana) para prevenir lecturas erróneas. (coherente con 29119-3).

Bloques recomendados en el tablero (Grafana/BI):

- **Arriba:** tarjetas **SLO** (disponibilidad, p95) y **DORA** (DF, LT, CFR, MTTR).
- **Centro:** DER por severidad y **tendencias** (últimas 4–8 semanas).
- **Abajo:** *drill-down* a rutas/endpoints, *heatmaps* de latencia y vínculo a **trazas** (*exemplars*).

V. Cadencia y canales (sincrónico + asíncrono)

- **Asíncrono:** *one-pager* (BLUF + gráfico principal) antes de la reunión. **Versiona** en el repositorio del proyecto con fecha y *tag* del *release* (cumple 29119-3).
- **Sincrónico:** *readout* de 15–30 min. El **QA Lead** presenta; **PO** valida riesgo/valor; dudas técnicas al **Dev/SRE**. En incidentes, usa el **Communications Lead** SRE para actualizaciones periódicas.

VI. Ejemplos

- **Resumen ejecutivo (BLUF):**

BLUF: *GO con guardas.* La versión **v3.4.2** cumple **SLO** (p95 *checkout* 280 ms, objetivo 300 ms) y **DER** 1.6 % (< 2 %).

Riesgos: saturación ocasional del *pool* de DB (picos 18:00–19:00).

Acciones: aumentar *pool* a 64, *canary* 25 % por 24 h, alerta por **burn-rate** > 2×/1h.

Tabla 3. RAG (criterios declarados):

Riesgo	Criterio	Estado	Mitigación	Dueño
p95 /checkout	>300 ms por 2h (prom 5m)	🟡	*Canary y autoscaling*	SRE
DER	≥ 2 % sev. Major+	🟢	-	QA
CFR post-deploy	>15 %	🟢	*Feature flag OFF*	PO

Fuente: autoría propia.

Asegurar que la leyenda y umbrales estén visibles en el reporte/slide.

VII. Antipatrones (y cómo evitarlos)

- **PDF de 20 páginas sin conclusión** → aplicar **BLUF/Minto** y limita el **cuerpo** a 1–2 gráficas clave.
- **Gráficos bonitos, difíciles de leer** → usar diseños que aprovechen **procesamiento pre-atencional** y evita ***chartjunk***.
- **Definiciones opacas** (¿qué es DER/MTTR?) → incluye **fórmula y ventana** junto al KPI (29119-3).
- **Quien repara comunica** (silencio externo) → separar **remediación** de **comunicación** (rol de **Communications Lead**).

c. Roadmap de optimización incremental

Un *roadmap* de optimización incremental guía una secuencia **pequeña-pero-constante** de mejoras que elevan la **calidad del producto**, la **fluidez de entrega** y la **fiabilidad operativa**, con base en métricas y experimentos de corto alcance (Kaizen/PDSA), en políticas de **SLO/error-budget** y en enfoques de **foco en el cuello de botella** (TOC). De este modo se prioriza lo que más impacta resultados (valor y riesgo) y se evita la **transformación big-bang.**

I. Poner "el piso" de medición (línea base y *north star*)

- **Definir la línea base** con KPIs de entrega (**DORA: frecuencia de despliegue, *lead time*, tasa de fallos, MTTR**) y de calidad/operación (DER, *escape rate*, cumplimiento de **SLO**). Usar la taxonomía DORA para medir *throughput* y *stability*.



- **Escribir los SLO** (p. ej., *p95 checkout* ≤ 300 ms, disponibilidad 99.9 %) y su **política de *error-budget*** (qué se detiene y quién decide ante sobreconsumo). Estas reglas convierten la fiabilidad en decisiones operables del *roadmap*.

II. Mapear el flujo y hallar el cuello de botella

- ***Value Stream Mapping* (VSM):** visualizar flujo desde idea → producción; captura tiempos de espera, *handoffs* y puntos de fricción. El VSM facilita conversaciones de mejora y priorización basada en **dónde se atasca** el flujo.

- ***Theory of Constraints* (TOC):** aplica los **cinco pasos de enfoque** (*identify–exploit–subordinate–elevate–repeat*) para intervenir **una** restricción a la vez (evita "optimizar" lo que no limita el sistema).

III. Priorizar con economía de decisiones

- Ordenar el ***backlog* de mejoras** (no de *features*) por **impacto** y **urgencia** usando ***Cost of Delay*** (CoD): cuánto valor se pierde por cada semana/mes sin esa mejora. Este lenguaje común reduce debates subjetivos y acelera acuerdos de inversión.

- Conectar el *roadmap* a **OKRs** (objetivos y resultados clave) para alinear a negocio/tecnología y revisar progreso trimestralmente.

IV. Diseñar mejoras como experimentos PDSA

Cada iniciativa del *roadmap* se formula como **hipótesis → experimento → métrica de éxito → estandarización** si funciona (*Plan–Do–Study–Act*). Esto convierte la estrategia en aprendizaje medible (no opiniones).

Ejemplo:

Hipótesis: **Si activamos caché de catálogo, el p95 de /items cae ≥ 20 % sin elevar error-rate.**
Experimento: canary 5 % usuarios por 24 h con *feature flag.*
Éxito: p95 ↓ ≥ 20 % y *burn-rate* < 2×.
Act: estandarizar y retirar *flag* en 7 días.

V. Ejecutar con despliegues seguros (canary/flags) y *gates*

- **Canary** y *feature flags* permiten validar en pequeño, con criterios de éxito ligados a SLO/error-budget (si el **burn-rate** se dispara, pausa/revierte). Integra estos criterios como *gates* previos a subir el porcentaje.

- Mantener *gates* de calidad en CI/CD (cobertura mínima en código nuevo, 0 *critical*, SAST limpio) y automatizar el **bloqueo** si no se cumplen. (Ver unidad sobre *quality gates*).

VI. Medir el efecto y hacer visible el progreso

- Tablero con **tarjetas KPI:** DORA, DER/escape-rate, SLO (*p95/p99*, *error-rate*), y tendencia de *burn-rate*. Evitar promedios; usar percentiles y ventanas claras.
- Para mejoras de **escalabilidad**, modelar la capacidad con **USL (Universal Scalability Law)** y proyectar el *ceiling* al crecer usuarios/concurrencia (evitar extrapolaciones lineales ingenuas).

VII. Cadencia de gobierno (ligera y efectiva)

- **Semanal:** *stand-up* de mejora (15 min) para estado de experimentos y decisiones **seguir/pausar/descartar**.
- **Mensual:** revisión de *roadmap:* nuevos cuellos de botella (TOC), CoD actualizado y **OKRs**.
- **Trimestral:** *readout* ejecutivo: deltas en DORA/SLO y retorno de mejoras (tiempo ahorrado, reducción de incidentes).

VIII. Ejemplo de roadmap (90 días)

Tabla 4. Ejemplo de *roadmap*

Semana	Iniciativa (hipótesis)	Tipo	Métrica objetivo	Guardia/Decisión
1-2	VSM de *checkout* y *release*	Flujo	Identificar 1 cuello de botella	TOC paso 1–2
3-4	*Canary* caché catálogo 5 %	Perf	p95 /items –20 %	SLO *burn-rate* < 2× → escalar
5-6	Reducir *batch size* MR (≤ 300 LOC)	Entrega	↑ despliegue/semana + ↓ CFR	OKR de *flow* (DORA)
7-8	Gate de dependencias (SCA)	Calidad	0 críticas en MR	Gate bloquea merge
9-10	*Load-shedding* en picos	Fiabilidad	MTTR –20 %	Error-budget policy activa
11-12	Refuerzo del cuello elev. (DB pool)	Escala	Curva USL +15 % cap.	Validación con USL

Fuente: autoría propia.

IX. Antipatrones (y cómo evitarlos)

- **Mejorar todo a la vez** → aplicar **TOC** y atacar **1** restricción por ciclo.
- ***Roadmaps* sin economía** → priorizar con ***Cost of Delay*** y visibilizar el costo de esperar.
- **Medir solo promedios** → usar **percentiles** y ventanas; los SLO se violan en colas largas, no en promedios.
- ***Canary* sin criterios** → definir el éxito en términos de **SLO/burn-rate** antes de escalar exposición.
- **OKRs decorativos** → revisar mensualmente y conectar cada KR con una **métrica visible** del tablero.

Revisa tus aprendizajes

Para comprobar lo aprendido en esta unidad, realiza el siguiente ejercicio respondiendo a las preguntas que se presentan a continuación. Recuerda que no debes subir tus respuestas a la plataforma; compártelas con el docente y tus compañeros durante el encuentro sincrónico.

1. ¿Cuál afirmación describe mejor el ***escape rate*** (DER)?

a. Defectos encontrados por KLOC en pruebas unitarias.
b. Porcentaje de defectos hallados en QA respecto al total reportado.
c. Porcentaje de defectos que llegan a producción respecto del total detectado en el ciclo.
d. Tiempo promedio entre fallos en producción.

2. En las métricas DORA, ¿qué par evalúa principalmente “estabilidad” de la entrega?

a. DF y LT.
b. CFR y MTTR.
c. DF y CFR.
d. LT y DER.

3. Para comunicar latencia en un tablero ejecutivo, la práctica recomendada es mostrar…

a. Promedio simple (avg).
b. Mediana únicamente.
c. Percentiles (p95/p99) y línea de SLO.
d. Máximo observado.



4. En el flujo de defectos, la transición correcta antes de “Closed” es…

   a. New → Closed.
   b. Assigned → Closed.
   c. Resolved (con *Resolution*) → Verified → Closed.
   d. Triaged → Closed.

5. Diferencia clave entre ***severity y priority:***

   a. *Severity* es negocio; *priority* es técnico.
   b. “*Severity*” es impacto técnico; “*priority*” es urgencia de negocio.
   c. Ambas son sinónimos.
   d. Se determinan sólo por el order book.

6. En la reunión de triage, el rol típicamente ***Accountable*** de la priorización es…

   a. QA Lead.
   b. Dev Lead.
   c. Product Owner.
   d. Scrum Master.

7. Para prevenir que una regresión reaparezca tras un ***hotfix***, lo más importante es…

   a. Escribir un documento de lecciones aprendidas.
   b. Agregar una prueba automatizada que primero falla y luego pasa con el fix (“golden regression”).
   c. Aumentar el tamaño del pool de conexiones.
   d. Hacer *rollback* siempre.

8. ¿Cuál es el orden recomendado al aplicar técnicas de RCA (enfoque didáctico de la unidad)?

   a. 5 Whys → Pareto → Ishikawa.
   b. Ishikawa (divergir) → Pareto (priorizar) → 5 Whys (profundizar).
   c. Pareto → Ishikawa → 5 Whys.
   d. 5 Whys → Ishikawa → Pareto.

9. Un post-mortem ***“blameless”*** debe…

   a. Identificar responsables individuales del error.
   b. Minimizar detalles técnicos para evitar sesgos.
   c. Enfocarse en condiciones del sistema y acciones SMART, no en culpas.
   d. Cerrarse sin evidencias si ya se restauró el servicio.



10. ¿Qué es un ***quality gate*** en CI/CD?

- **a.** Un tablero de BI con KPIs.
- **b.** Un conjunto de condiciones objetivas que bloquea el avance si no se cumplen (cobertura, SAST, Sonar, etc.).
- **c.** Un *feature flag* de producción.
- **d.** Un reporte mensual de QA.

11. La política ***fail-fast*** bien aplicada en la pipeline implica…

- **a.** Ejecutar primero las pruebas más lentas.
- **b.** Fallar temprano con etapas baratas (lint/unit/SAST) para ahorrar ciclos.
- **c.** Nunca abortar ramas paralelas.
- **d.** Posponer análisis estático al final.

12. Los ***feature toggles*** aportan principalmente…

- **a.** Cobertura de código más alta.
- **b.** Desacoplar despliegue de exposición para *rollouts* graduales y *kill switches*.
- **c.** Sustituir los *quality gates*.
- **d.** Evitar necesidad de monitoreo.

13. En comunicación ejecutiva (BLUF), lo primero que debe verse es…

- **a.** El anexo técnico completo.
- **b.** Un gráfico de Pareto.
- **c.** La conclusión y la recomendación, seguidas de la evidencia.
- **d.** La cronología detallada del incidente.

14. Para priorizar el ***roadmap*** de optimización incremental, la técnica destacada es…

- **a.** MoSCoW sobre *features.*
- **b.** Cost of Delay (valor perdido por esperar) y foco TOC en el cuello de botella.
- **c.** FIFO por orden de llegada.
- **d.** Voto a mano alzada.

Lecciones aprendidas

En esta lección integraste un marco completo para dirigir la calidad con datos y convertirla en decisiones de producto. Aprendiste a definir e interpretar KPIs de QA (defect density, *escape rate*, MTTR) y métricas DORA para equilibrar velocidad y estabilidad, y a plasmarlas en dashboards ejecutivos con percentiles y SLO/SLA visibles.

Estandarizaste el ciclo de vida del defecto (New → Closed), la dinámica de triage y la distinción severity/priority para priorizar con impacto de negocio. Profundizaste en análisis causa-raíz (Ishikawa → Pareto → 5 Whys) y en la conducción de post-mortems blameless con acciones SMART.

Finalmente, conectaste la calidad al flujo de entrega mediante quality gates, fail-fast y feature toggles, y consolidaste una práctica de mejora continua (Kaizen, retros, comunicación efectiva) apoyada en un *roadmap* de optimización incremental de 90 días. Con estas piezas, puedes medir, priorizar, mejorar y sostener la calidad de forma sistemática en tu organización.

Referencias bibliográficas

- Apdex Alliance. (2007). *Apdex Technical Specification v1.1.* **Disponible aquí.**
- Atlassian. (2025). *How to modify your workflow to allow issues to be reopened; Resolved issues appearing in Open issues filters.* **Disponible aquí.**
- Bugzilla / The Document Foundation. (2024). *QA/Bugzilla – Status/RESOLVED; Bug workflow.* **Disponible aquí; Disponible aquí.**
- DORA. (2025). *DORA's software delivery metrics: the four keys.* **Disponible aquí.**
- GCC Bugzilla. (s. f.). *Bug fields – Resolution (DUPLICATE, etc.).* **Disponible aquí.**
- GitHub—DORA Team. (2024). *Four Keys (open source pipeline for DORA).* **Disponible aquí.**
- Google Cloud. (2020). *Using the Four Keys to measure your DevOps performance.* **Disponible aquí.**
- Google Cloud. (2021). *Announcing DORA 2021 Accelerate State of DevOps report.* **Disponible aquí.**
- Google SRE. (2021). *Incident Metrics in SRE (article & PDF).* **Disponible aquí.**
- Google SRE. (s. f.). *The Art of SLOs / SRE Workbook.* **Disponible aquí.**
- Grafana Labs. (2020). *How to visualize Prometheus histograms in Grafana.* **Disponible aquí.**
- Grafana Labs. (2024). *Best practices to design your first dashboard.* **Disponible aquí.**
- Grafana Labs. (s. f.). *Dashboard best practices (docs).* **Disponible aquí.**
- Grafana Labs. (s. f.). *Introduction to exemplars.* **Disponible aquí.**
- IEEE. (1998). *IEEE-1061: Software Quality Metrics Methodology (overview/order page).* **Disponible aquí.**
- IEEE. (2009). *IEEE 1044-2009 – Standard classification for software anomalies.* **Disponible aquí.**
- ISO/IEC. (2016). *ISO/IEC 25023: Measurement of system and software product quality.* **Disponible aquí.**
- ISO/IEC/IEEE. (2013). *29119-3: Software testing — Test documentation (plantillas de incidentes).* **Disponible aquí;** ejemplos académicos.

- ISO/IEC/IEEE. (2021). *29119-2: Software testing — Test processes.* **Disponible aquí.**
- Microsoft. (2024). *PERCENTILEX.INC (DAX); PERCENTILE.INC (DAX).* **Disponible aquí.**
- Prometheus Authors. (s. f.). *Histograms and summaries; Querying basics; Query examples.* **Disponible aquí.**


UNIVERSIDAD DE
LA SALLE
Vigilada MinEducación